1.

Aim:

To design and implement a fault-tolerant network topology using cisco Packet Tracer with 6 system and 2 switches, using redundant links and to verify that communication continues when one network link fails.

Procedure:

- Open cisco Packet Tracer
- Place 6 PC in the workspace
- Place 2 switches
- Connect the PCs to the switches using copper straight-through cables
- Add redundant connect between two switches using multiple links
- Assign IP address to all 6 PCs
- Ensure that all connections are active and wait for the link to turn green.
- Use the command prompt of a PC and execute.
- Verify that packets are successfully transmitted.

* Disconnect or shut down one of the redundant links between the switches
* Again perform the ping
* Observe that communication continues through the remaining redundant path.

Result:

The EIGRP was successfully configured using an access point and connectivity

The fault - tolerant network was successfully configured and communication continued even after one link failed.

#7

Aim:

To configure a wireless local area network using cisco packet tracer with one access point and five wireless-enable systems and to verify connectivity between the wireless client

Procedure:

- Open cisco packet tracer
- Place one Access Point in the work space
- Place 5 wireless enabled devices ~~such as~~ like lap
- Configure the Access point with a suitable SSID for example
  SSID: WLAN
- Configure wireless security if required
  Security: WPA2-PSK
  Password: network123
- Enter the configured password
- Assign IP addresses to five wireless system
- Open the command Prompt of an laptop
- ping the other wireless client

Result:

The WLAN successfully configured using an Access Point and connectivity between all 5 wireless system was successfully verified.

communication continued even after one link failed.

3.

Aim:

To capture and analyze ICMP packets using Wireshark and study their header fields and payload.

Procedure:

* Open wireshark on the device.
* Select the active network interface through which packets are being transmitted.
* Start the packet capture by clicking the start button.
* Generate or capture multicast traffic on the network.
* Stop the capture after sufficient packets are collected.
* Apply display filter "icmp"
* It will display only packets related to the internet ICMP.
* Select any ICMP Packet from the captured packet list

4) Expand IGMP.

5) Observe type, maximum response time, checksum, group address.

Sample output:

Protocol: IGMP

Type: Membership Report

Checksum: Valid

Group Address: Multicast Address.

Result:

IGMP Packets were successfully captured using Wireshark and their header fields and payload information were analysed.

[illegible]

Result:

Thus the C program was successfully executed using [illegible] [illegible] and [illegible] implemented and then [illegible] [illegible] were verified.

Aim:

To implement and demonstrate the [illegible] using C programming for suitable data representation.

Procedure:

1) Open a C programming environment such as Turbo C, VS code, GCC.

2) Create a new C source file named [illegible]

3) Write a program to implement Stack and test methods.

4) The program first accepts the number of [illegible]

4.

Aim:

To implement and demonstrate the Stop and wait protocol using C Programming for Reliable data transmission.

Procedure:

*) open a C programming environment such as turbo C, VS code or GCC.

*) Create a new C source file named

*) Create a Write a program to implement the Stop-and-Wait protocol.

*) the program first accepts the number of frames

transmitted.

1) The sender transmits one frame at a time.
2) After sending a frame, the sender waits for an ACK
3) If the ACK received, the sender to transmit the next frame.
4) If the ACK is not received, sender retransmits the same frame.
5) This process is repeated until all frames are successfully transmitted.
6) Compile and execute the program

Code:

```
#include<stdio.h>
int main(){
  int n,i; char a;
scanf("%d",&n);
for(i=1;i<=n;i++){
printf("Frame %d ACK(y/n):",i);
scanf("%c",&a);
if(a=='n') i--;
}
printf("Done");
}
```

Sample output:

Frame 1 ACK(y/n): y
Frame 2 ACK(y/n): n
Frame 2 ACK(y/n): y
Frame 3 ACK(y/n): y

Done.

Result:
The stop-and-wait was successfully implemented using C progr
the reliable data transmission using acknowledgement and

to be transmitted

1) The sender transmits one frame at a time.
2) After sending a frame, the sender waits for an ACK
3) If the ACK received, the sender to transmit the next frame.
4) If the ACK is not received, sender retransmits the same frame.
5) This process is repeated until all frames are successfully transmitted.
6) Compile and execute the program

Code:

```
#include<stdio.h>
int main(){
  int n, i; char a;
scanf("%d", &n);
for(i=1; i<=n; i++){
printf("Frame %d ACK (y/n):", i);
scanf("%c", &a);
if (a=="n") i--;
}
printf("done");
}
```

Sample output:

Frame 1 ACK (y/n): y
Frame 2 ACK (y/n): n
Frame 2 ACK (y/n): y
Frame 3 ACK (y/n): y

Done.

Result:

The Stop-and-Wait was successfully implemented using C program